Lab 10: Fixed Effects, DID, and the Staggered-Adoption Problem

SOCIOL 723, Social Statistics II, Thursday, October 29

Wenhao Jiang

Table of contents

1 Goals

1. Confirm that within-transformation, first differences, and unit dummies are the same estimator.
2. Estimate a canonical 2×2 DID and its event-study generalization.
3. **Watch two-way fixed effects fail on staggered data where we know the truth.**
4. Decompose TWFE into its 2×2 pieces (Goodman-Bacon).
5. Use Sun–Abraham and Callaway–Sant'Anna to get the right answer.

6. Bound how badly parallel trends could fail.

```
library(dplyr)
library(ggplot2)
library(fixest)
library(did)
library(bacondecomp)

data(base_stagg, package = "fixest")
stg <- base_stagg |> as.data.frame() |>
  mutate(D = as.integer(time_to_treatment >= 0),
         cohort = year_treated) |>            # 10000 codes never-treated
  select(id, year, y, D, cohort, time_to_treatment, x1, treatment_effect_true)

c(units = n_distinct(stg$id), periods = n_distinct(stg$year),
  cohorts = n_distinct(stg$cohort),
  never_treated_units = sum(!duplicated(stg$id) & stg$cohort == 10000))
#>                 units              periods             cohorts never_treated_units
#>                    95                   10                  10                  50
```

The simulated panel has 100 units over 10 periods. Half are never treated; the other half adopt in staggered fashion, one cohort per period.

2 Fixed effects: three routes, one estimator

```
d <- stg

## 1. unit dummies
m_dummy  <- lm(y ~ D + factor(id) + factor(year), data = d)

## 2. within transformation, by hand
## y_it - ybar_i - ybar_t + ybar
dm <- d |>
  group_by(id)   |> mutate(y_i = y - mean(y), D_i = D - mean(D)) |>
  group_by(year) |> mutate(y_t = mean(y_i),  D_t = mean(D_i)) |>
  ungroup() |>
  mutate(y_w = y_i - y_t, D_w = D_i - D_t)   # grand mean already removed by y_i
m_within <- lm(y_w ~ D_w, data = dm)

## 3. fixest
m_fe <- feols(y ~ D | id + year, data = d)

round(c(dummies = coef(m_dummy)["D"],
```

```r
        within   = coef(m_within)["D_w"],
        fixest   = coef(m_fe)["D"]), 6)
#>  dummies.D within.D_w   fixest.D
#>     -3.468     -3.468     -3.468
```

All three agree to six decimals. This is the Frisch–Waugh–Lovell theorem from Week 1: regressing on unit dummies *is* demeaning within unit.

```r
## how much variation survives the transformation?
c(sd_D_raw = sd(d$D), sd_D_within = sd(dm$D_w))
#>    sd_D_raw sd_D_within
#>      0.4254      0.2531
```

That ratio is the price of fixed effects. Everything the estimator knows comes from the smaller number.

3 A clean 2×2 DID

Restrict to two cohorts and two periods so the design is transparent.

```r
d22 <- stg |>
  filter(cohort %in% c(5, 10000), year %in% c(4, 6)) |>
  mutate(treated = as.integer(cohort == 5),
         post    = as.integer(year == 6))

cellmeans <- d22 |> group_by(treated, post) |> summarise(m = mean(y), .groups = "drop")
knitr::kable(tidyr::pivot_wider(cellmeans, names_from = post, values_from = m,
                                names_prefix = "post_"), digits = 3)
```

treated	post_0	post_1
0	-0.423	0.032
1	0.856	-2.822

```r
cm <- with(cellmeans, setNames(m, paste0(treated, post)))
did_hand <- (cm["11"] - cm["10"]) - (cm["01"] - cm["00"])

did_reg <- coef(lm(y ~ treated * post, data = d22))["treated:post"]
c(by_hand = did_hand, regression = did_reg)
#>              by_hand.11 regression.treated:post
#>                  -4.132                  -4.132
```

The interaction coefficient *is* the difference in differences.

4 The staggered problem

`base_stagg` is simulated, so we know the answer. The column `treatment_effect_true` records each observation's true effect.

```
true_att <- mean(stg$treatment_effect_true[stg$D == 1])
c(true_ATT = true_att)
#> true_ATT
#>       -1
```

```
stg |> filter(D == 1) |>
  group_by(time_to_treatment) |>
  summarise(truth = mean(treatment_effect_true)) |>
  ggplot(aes(time_to_treatment, truth)) +
  geom_line(colour = "navy", linewidth = .9) + geom_point(colour = "navy") +
  labs(x = "periods since treatment", y = "true effect") +
  theme_minimal(base_size = 10)
```

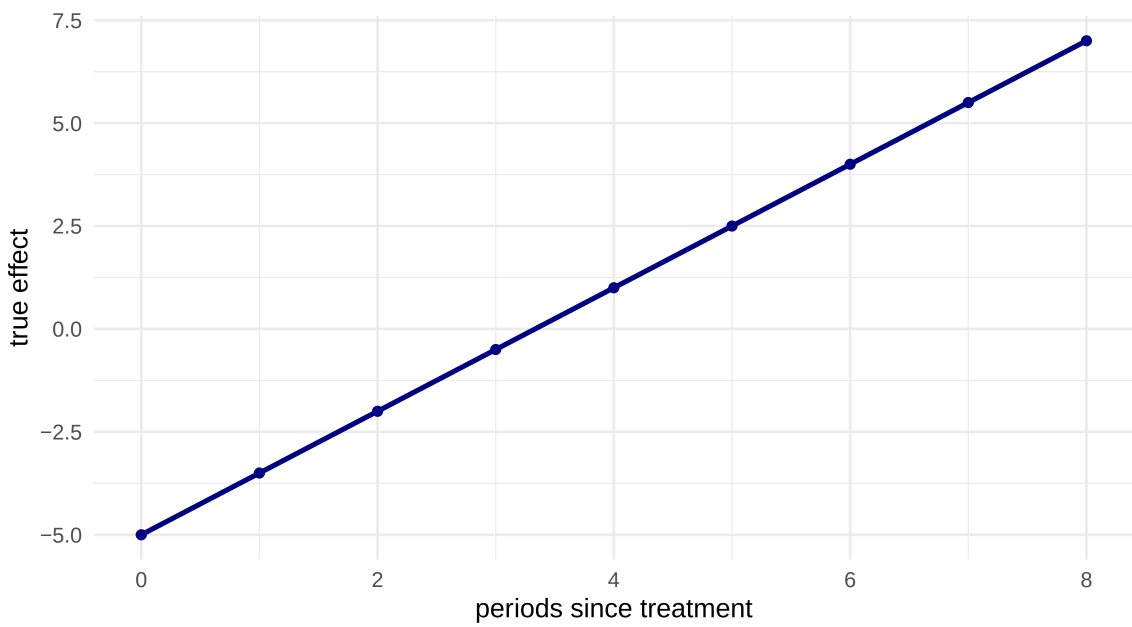

Figure 1: The true effect by time since treatment. It grows in magnitude with exposure — exactly the dynamic pattern that breaks TWFE.

4.1 What TWFE reports

```
m_twfe <- feols(y ~ D | id + year, data = stg, cluster = ~ id)
c(TWFE = coef(m_twfe)["D"], truth = true_att)
#> TWFE.D  truth
#> -3.468 -1.000
```

> **!** Important
>
> The true ATT is -1. TWFE reports roughly -3.5 — **three and a half times too large**. Nothing is misspecified in the usual sense: there is no omitted variable, no functional-form error, no measurement problem. The failure comes entirely from using already-treated units as controls while their own effects are still growing.

4.2 Where the damage comes from

```r
bd <- bacon(y ~ D, data = stg, id_var = "id", time_var = "year")
#>                        type weight avg_est
#> 1 Earlier vs Later Treated 0.1429  -2.460
#> 2 Later vs Earlier Treated 0.1429  -8.485
#> 3     Treated vs Untreated 0.7143  -2.666

bd |> group_by(type) |>
  summarise(total_weight = sum(weight),
            weighted_estimate = sum(estimate * weight) / sum(weight),
            n_comparisons = n(), .groups = "drop") |>
  knitr::kable(digits = 4)
```

type	total_weight	weighted_estimate	n_comparisons
Earlier vs Later Treated	0.1429	-2.460	36
Later vs Earlier Treated	0.1429	-8.485	36
Treated vs Untreated	0.7143	-2.666	9

Read the `Earlier vs Later Treated` and `Later vs Earlier Treated` rows. The second is the **forbidden comparison** — already-treated units serving as controls. Its weight tells you how much of the TWFE estimate is contaminated.

```r
ggplot(bd, aes(weight, estimate, colour = type, shape = type)) +
  geom_point(size = 2, alpha = .8) +
  geom_hline(yintercept = coef(m_twfe)["D"], linetype = 2) +
  geom_hline(yintercept = true_att, colour = "firebrick") +
  labs(x = "weight", y = "2x2 DID estimate", colour = NULL, shape = NULL) +
  theme_minimal(base_size = 9) + theme(legend.position = "bottom")
```

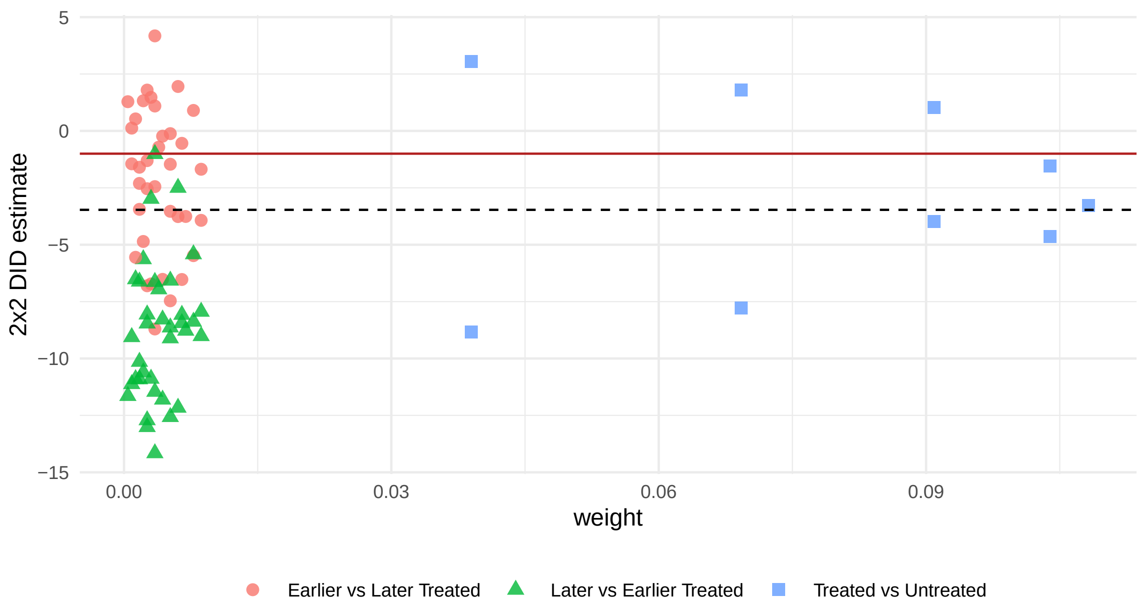

Figure 2: Goodman-Bacon decomposition: each point is a 2x2 comparison, positioned by its weight and its estimate.

The dashed line is the TWFE estimate — a weighted average of everything shown. The red line is the truth.

5 Estimators that work

5.1 Sun–Abraham

```
m_sa <- feols(y ~ sunab(cohort, year) | id + year,
              data = stg, cluster = ~ id)
summary(m_sa, agg = "att")
#> OLS estimation, Dep. Var.: y
#> Observations: 950
#> Fixed-effects: id: 95,  year: 10
#> Standard-errors: Clustered (id)
#>     Estimate Std. Error t value Pr(>|t|)
#> ATT  -0.7552     0.4211  -1.793 0.076144 .
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> RMSE: 1.98883     Adj. R2: 0.479262
#>                 Within R2: 0.424684
```

```r
iplot(m_sa, main = "", xlab = "periods since treatment")
```

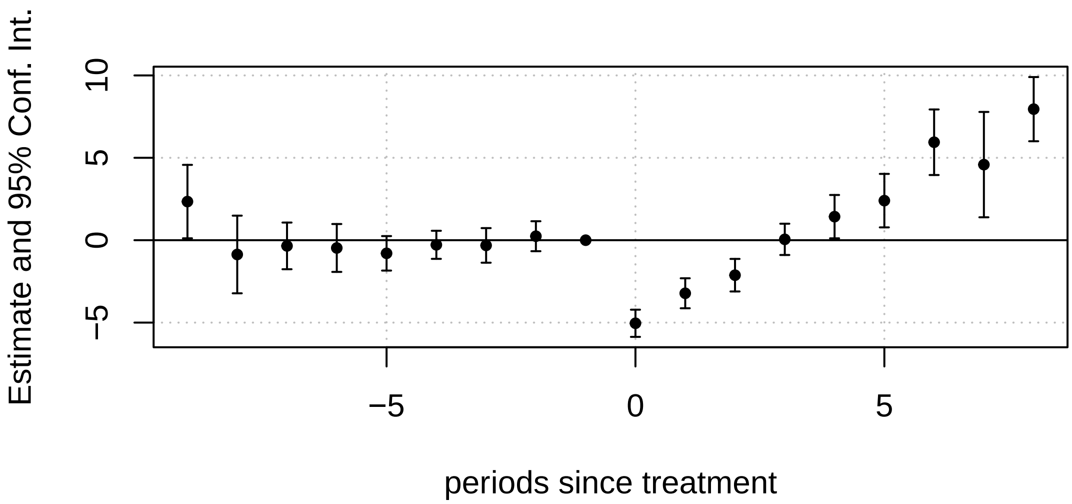

Figure 3: Sun–Abraham event study. Pre-period coefficients hover around zero; post-period coefficients trace the true dynamic path.

5.2 Callaway–Sant'Anna

```r
set.seed(723)
cs <- att_gt(yname = "y", tname = "year", idname = "id",
             gname = "cohort", data = stg,
             control_group = "notyettreated", bstrap = TRUE, cband = FALSE)
cs_att <- aggte(cs, type = "simple", na.rm = TRUE)
summary(cs_att)
#>
#> Call:
#> aggte(MP = cs, type = "simple", na.rm = TRUE)
#>
#> Reference: Callaway, Brantly and Pedro H.C. Sant'Anna.  "Difference-in-Differences with Mu
#>
#>
#>      ATT    Std. Error     [ 95%  Conf. Int.]
#>  -0.7995        0.7115     -2.194      0.5949
#>
#>
#> ---
#> Signif. codes: `*' confidence band does not cover 0
#>
#> Control Group:  Not Yet Treated,  Anticipation Periods:  0
#> Estimation Method:  Doubly Robust
```

```r
cs_dyn <- aggte(cs, type = "dynamic", na.rm = TRUE)
ggdid(cs_dyn) + theme_minimal(base_size = 9)
```

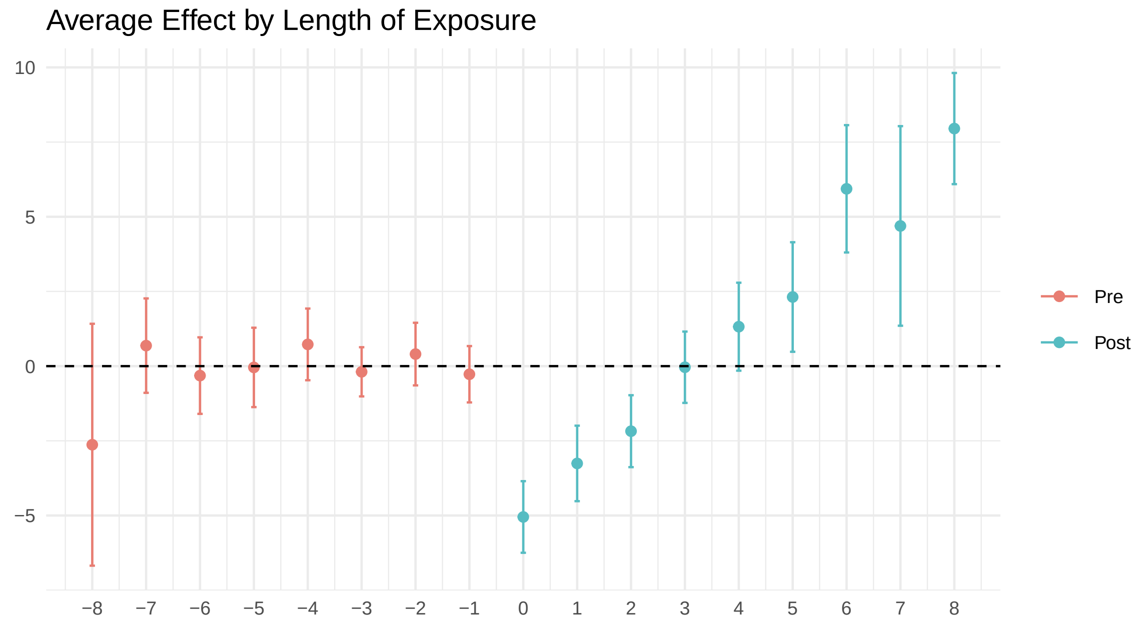

Figure 4: Callaway–Sant'Anna dynamic effects, aggregated across cohorts by time since treatment.

5.3 The scoreboard

```r
tibble::tibble(
  estimator = c("Truth", "TWFE", "Sun-Abraham", "Callaway-Sant'Anna"),
  estimate = c(true_att, coef(m_twfe)["D"],
               summary(m_sa, agg = "att")$coeftable[1, 1],
               cs_att$overall.att)
) |> knitr::kable(digits = 4)
```

estimator	estimate
Truth	-1.0000
TWFE	-3.4676
Sun-Abraham	-0.7552
Callaway-Sant'Anna	-0.7995

Sun–Abraham and Callaway–Sant'Anna land near the truth of -1. TWFE does not, and TWFE is the specification that appears in thousands of published papers.

(The two modern estimators are not exactly -1 because the "simple" aggregation weights cohorts and periods differently from a naive average over treated observations. That is a choice of estimand, not an error — see Exercise 3.)

6 A real application

`did::mpdta` contains county-level teen employment and the timing of minimum-wage increases, 2003–2007.

```
data(mpdta, package = "did")
c(counties = n_distinct(mpdta$countyreal), years = n_distinct(mpdta$year),
  cohorts = paste(sort(unique(mpdta$first.treat)), collapse = ", "))
#>                counties                         years                  cohorts
#>                   "500"                           "5" "0, 2004, 2006, 2007"
```

```
mpdta |>
  group_by(first.treat, year) |>
  summarise(y = mean(lemp), .groups = "drop") |>
  ggplot(aes(year, y, colour = factor(first.treat))) +
  geom_line(linewidth = .8) + geom_point() +
  labs(x = NULL, y = "mean log teen employment", colour = "cohort") +
  theme_minimal(base_size = 10)
```

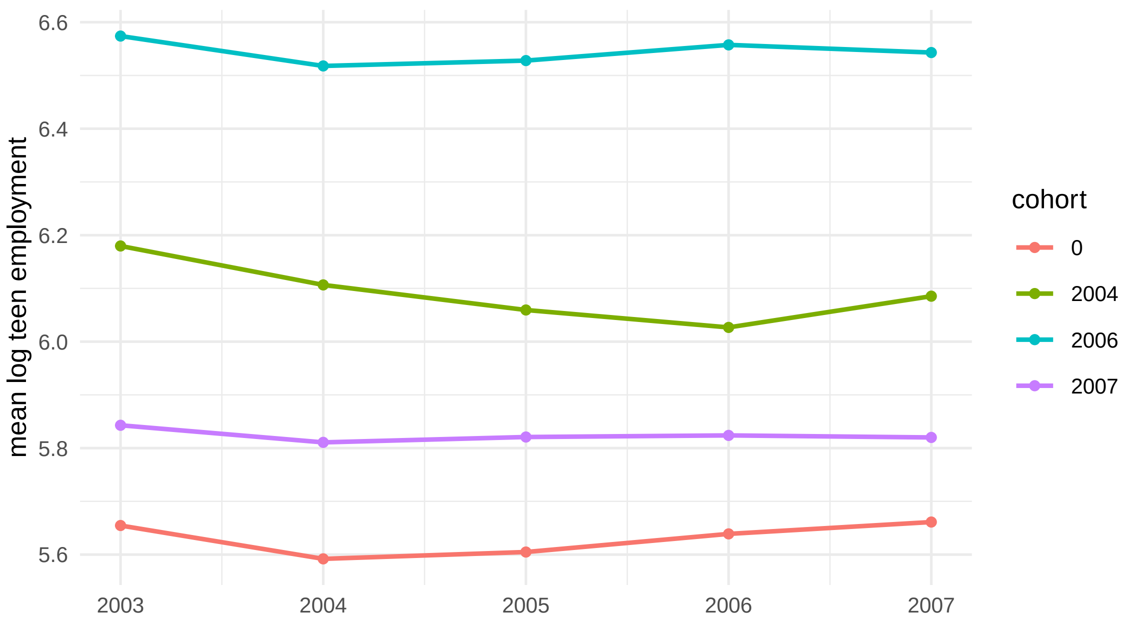

Figure 5: Raw group means of log teen employment by treatment cohort. Always draw this before any regression.

```
## `treat` in mpdta is a time-invariant cohort flag; build the time-varying
## treatment indicator ourselves
mp <- mpdta |> mutate(D = as.integer(first.treat > 0 & year >= first.treat))

m_twfe_mp <- feols(lemp ~ D | countyreal + year, data = mp,
                   cluster = ~ countyreal)
```

```r
set.seed(723)
cs_mp <- att_gt(yname = "lemp", tname = "year", idname = "countyreal",
                gname = "first.treat", xformla = ~ lpop, data = mp,
                control_group = "notyettreated", bstrap = TRUE)
cs_mp_att <- aggte(cs_mp, type = "simple", na.rm = TRUE)

c(TWFE = coef(m_twfe_mp)["D"], CS = cs_mp_att$overall.att)
#>   TWFE.D       CS
#> -0.03655 -0.04135
```

```r
ggdid(aggte(cs_mp, type = "dynamic", na.rm = TRUE)) + theme_minimal(base_size = 9)
```

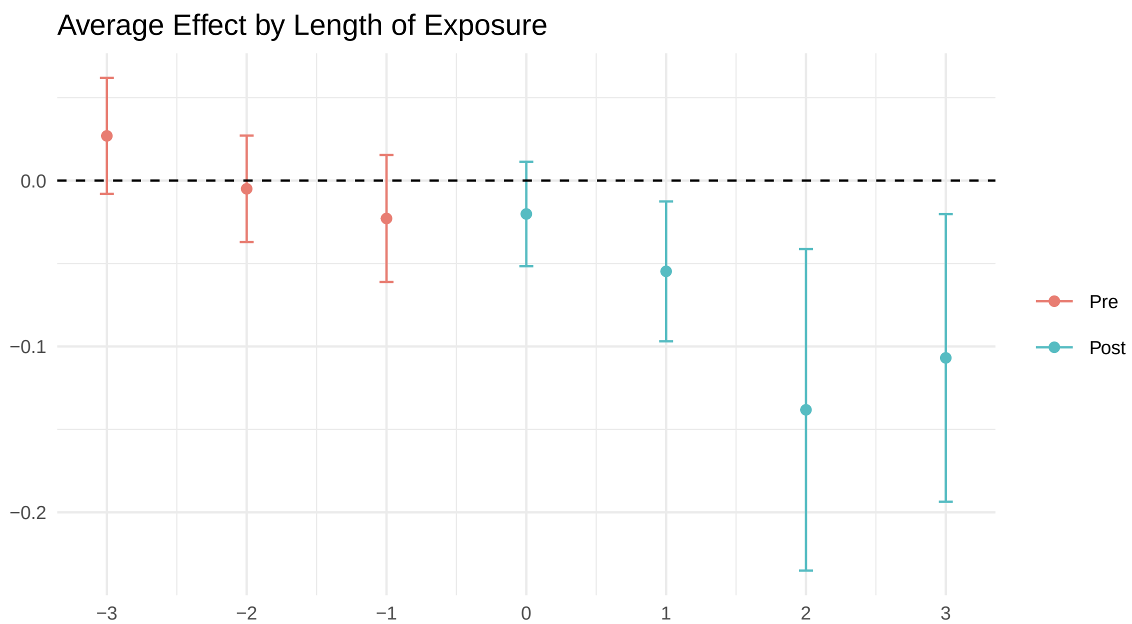

Figure 6: Event-study estimates for the minimum-wage application, with conditional parallel trends adjusting for county population.

Note the `xformla = ~ lpop` argument: this is **conditional** parallel trends, with the adjustment done by a doubly robust estimator. Week 6 has reappeared inside a DID.

7 How bad could parallel trends be?

Rather than testing pre-trends and proceeding, bound the violation.

```r
## A crude linear-extrapolation diagnostic -- NOT a Rambachan-Roth bound.
## It ignores sampling uncertainty and assumes the violation stays linear.
dyn <- aggte(cs_mp, type = "dynamic", na.rm = TRUE)
pre <- data.frame(e = dyn$egt, att = dyn$att.egt) |> filter(e < 0)
post_mean <- mean(dyn$att.egt[dyn$egt >= 0])
```

```r
## slope of the estimated pre-trend
pre_slope <- if (nrow(pre) > 1) coef(lm(att ~ e, data = pre))["e"] else NA
c(pre_trend_slope = pre_slope, mean_post_effect = post_mean,
  periods_to_explain = post_mean / pre_slope)
#>    pre_trend_slope.e     mean_post_effect periods_to_explain.e
#>             -0.02490             -0.07999              3.21299
```

Interpretation: if the untreated potential outcome had continued drifting at the estimated pre-trend slope, how many periods of drift would it take to account for the whole post-treatment effect? A small number means the design is fragile.

> ⚠ Warning
>
> This is a back-of-envelope diagnostic, not a formal sensitivity analysis. It treats the estimated pre-trend as if it were known, ignores uncertainty in both pieces, and assumes any violation is linear. For the real thing, use `HonestDiD` and report the **breakdown value** $\bar{M}$, how large the post-treatment violation may be, relative to the largest violation seen before treatment, before your conclusion reverses.

8 Exercises

1. **Build the failure yourself.** Simulate a staggered design with three cohorts and a *constant* treatment effect of 2. Show that TWFE recovers it. Then make the effect grow with exposure and show that TWFE does not. Explain the difference in one sentence.

2. **Not-yet-treated versus never-treated.** Re-run `att_gt()` with `control_group = "nevertreated"`. Compare with the not-yet-treated version. Which is more efficient, and under what assumption is each valid?

3. **Aggregation choices.** From the same `att_gt` object, compute `aggte(type = "group")` and `aggte(type = "calendar")`. These are three different estimands built from the same group-time effects. Write one sentence describing each in words.

4. **Pre-trend power.** For the `mpdta` application, simulate what a linear pre-trend of a given slope would do to the estimated pre-period coefficients. How large would the slope have to be before your pre-trend test detects it 80% of the time?

5. **Levels or logs?** `mpdta` supplies only log employment, so construct `emp <- exp(lemp)` and re-run the minimum-wage analysis in levels. Do the conclusions change? Which scale would you have pre-committed to, and why? (Recall from lecture that parallel trends in levels and in logs are different , and generally incompatible, assumptions.)

6. **Clustering.** Re-estimate the `mpdta` TWFE model clustering on state (`countyreal %/% 1000` gives the state FIPS code) rather than county. How many clusters are there, and what does Week 2 say you should do about it?

9 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] bacondecomp_0.1.1 did_2.3.0         fixest_0.12.1     ggplot2_4.0.0
#> [5] dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#>  [1] tidyselect_1.2.1     timeDate_4041.110    farver_2.1.2
#>  [4] S7_0.2.0             fastmap_1.2.0        TH.data_1.1-4
#>  [7] pROC_1.19.0.1        DRDID_1.2.3          caret_7.0-1
#> [10] digest_0.6.37        rpart_4.1.23         estimability_1.5.1
#> [13] timechange_0.3.0     lifecycle_1.0.4      dreamerr_1.5.0
#> [16] survival_3.6-4       magrittr_2.0.3       compiler_4.4.1
#> [19] rlang_1.3.0          tools_4.4.1          utf8_1.2.4
#> [22] yaml_2.3.10          data.table_1.18.2.1  knitr_1.48
#> [25] labeling_0.4.3       plyr_1.8.9           RColorBrewer_1.1-3
#> [28] multcomp_1.4-29      withr_3.0.1          purrr_1.0.2
#> [31] numDeriv_2016.8-1.1  stats4_4.4.1         nnet_7.3-19
#> [34] grid_4.4.1           fansi_1.0.6          xtable_1.8-4
#> [37] future_1.58.0        BMisc_1.4.8          emmeans_1.11.1
#> [40] globals_0.18.0       scales_1.4.0         iterators_1.0.14
#> [43] MASS_7.3-60.2        tinytex_0.53         cli_3.6.5
#> [46] mvtnorm_1.3-1        rmarkdown_2.28       generics_0.1.3
#> [49] bigmemory.sri_0.1.8  future.apply_1.20.0  reshape2_1.4.4
```

```
#> [52] stringr_1.5.1        splines_4.4.1        parallel_4.4.1
#> [55] fastglm_0.0.4        stringmagic_1.2.0    vctrs_0.6.5
#> [58] hardhat_1.4.2        Matrix_1.7-0         sandwich_3.1-1
#> [61] jsonlite_1.8.9       Formula_1.2-5        listenv_0.9.1
#> [64] trust_0.1-9          foreach_1.5.2        tidyr_1.3.1
#> [67] gower_1.0.2          recipes_1.3.1        bigmemory_4.6.6
#> [70] glue_1.7.0           parallelly_1.45.0    codetools_0.2-20
#> [73] stringi_1.8.4        lubridate_1.9.3      gtable_0.3.6
#> [76] tibble_3.2.1         pillar_1.9.0         htmltools_0.5.8.1
#> [79] ipred_0.9-15         lava_1.8.1           R6_2.5.1
#> [82] evaluate_1.0.0       lattice_0.22-7       class_7.3-22
#> [85] uuid_1.2-1           Rcpp_1.1.1           coda_0.19-4.1
#> [88] nlme_3.1-164         prodlim_2025.04.28   xfun_0.47
#> [91] ModelMetrics_1.2.2.2 zoo_1.8-12           pkgconfig_2.0.3
```